Ads Genie: Architecture Review & Strategic Recommendations

Prepared for

Render Analytics

Prepared by

Chris Morancie

Date

March 2026

Status: DRAFT For Discussion

This review is informed by the AI Production Framework. A structured methodology for building production-grade AI systems.

1. What You Got Right

The strategy document demonstrates strong product instinct. The bones are solid. The architecture needs rethinking but the foundational thinking is directionally correct across four key areas.

Competitive Read on SearchAtlas OTTO

OTTO is a generic tool built by an SEO company. Your institutional knowledge across **10+ verticals** and **4 years of MCC data** is a real moat, but, only if it's encoded in infrastructure, not prompts.

Phased Approach

The phased approach is directionally correct. Start simple, build toward a product. The sequencing instinct is sound even if the specific phases need revision.

Slack Integration as Differentiator

No competitor has this. But **how it's architected matters more than whether it exists.** The distinction between chat interface and control plane is critical.

Draft Mode / Human Review

Draft mode before campaign actions go live is **non-negotiable for production AI.** This is the right instinct and must be preserved in the revised architecture.

2.1 Build a Custom MCP Server. Don't Just Use Google's

The strategy document treats the MCP server as a pass-through pipe to the Google Ads API. This misses the most important opportunity in the entire project.

What Google's Official MCP Server Gives You

Two tools only: `search` (raw GAQL queries) and `list_accessible_customers`

Read-only. No write operations. No abstraction. No intelligence.

What MCP Is Actually For

Intent-to-API mapping. The MCP layer is where you encode what Render knows as callable tools, not raw API access, but your methodology made executable.

Example: What "Budget Waste" Means

When Kal says "which campaigns are wasting budget," that's not one GAQL query. It's a specific combination of signals that constitute *Render's definition* of waste:

Zero-Conversion Spend

Cost above threshold with zero conversions over N days

Irrelevant Search Terms

Search term reports showing irrelevant queries eating spend

Low Quality Scores

Quality scores below 5 dragging up CPC across the account

Misallocated Budget

Impression share lost to budget on campaigns that ARE converting, i.e. money in the wrong place

> That's Render's definition of waste. OTTO doesn't know this. A raw GAQL query doesn't encode this. But a custom MCP tool called `analyze_budget_waste` does and it does so deterministically, testably, every time.

Custom MCP Server: Proposed Tool Suite

The custom MCP server should expose the following tools, each encoding a specific piece of Render's institutional methodology:

Tool	What It Encodes
`analyze_budget_waste`	Render's methodology for identifying wasted spend, not a single metric, but a composite diagnostic combining cost thresholds, conversion data, search term quality, and impression share analysis
`diagnose_roas_drop`	Render's playbook: check search terms → auction insights → quality scores → landing pages → seasonality, in that specific order, with vertical-aware thresholds
`generate_negative_keywords`	Patterns learned across the entire MCC, not just one account but cross-client intelligence that no single-account tool can replicate
`health_check`	Render's definition of "healthy" per vertical. For example, a 2x ROAS for a day spa is different from a storage facility. Vertical-specific benchmarks baked in.
`benchmark_account`	Compare any account against Render's portfolio averages for that vertical. For example, the 4-year MCC dataset as a live benchmark engine
`draft_campaign`	Generate a campaign structure using Render's STAG methodology with vertical-specific defaults, not a generic template, but Render's actual approach
`cross_mcc_anomalies`	Surface accounts deviating from expected patterns. This is the cross-client intelligence OTTO can't do. The killer feature made systematic.

Why Methodology in Tools — Not Prompts

The Problem with System Prompts

→ **Prompts drift.** The LLM interprets methodology differently across sessions — no consistency guarantee.

→ **Prompts hallucinate.** "Check quality scores" in a prompt doesn't guarantee the right GAQL query runs.

→ **Prompts can't be unit tested.** MCP tools can, and should be.

→ **Prompts can't be versioned independently.** MCP tools can, with full change history.

The Core Principle

Methodology in a prompt = opinion.
Methodology in an MCP tool = infrastructure.

The LLM's job is to pick the right tool and interpret the results for the human. The tool's job is to execute Render's logic exactly. Every time.

This is the moat. Not a prompt. Not a model.
Infrastructure that encodes what you know.

2.2 Event-Driven Architecture: The Agent Should Be Always On

The strategy document describes a request-response system: someone asks a question in Slack, the agent queries Google Ads, returns an answer. **This is a chatbot with API access. It's not an agent.**

Current Design

"Ask me anything about your Google Ads accounts"

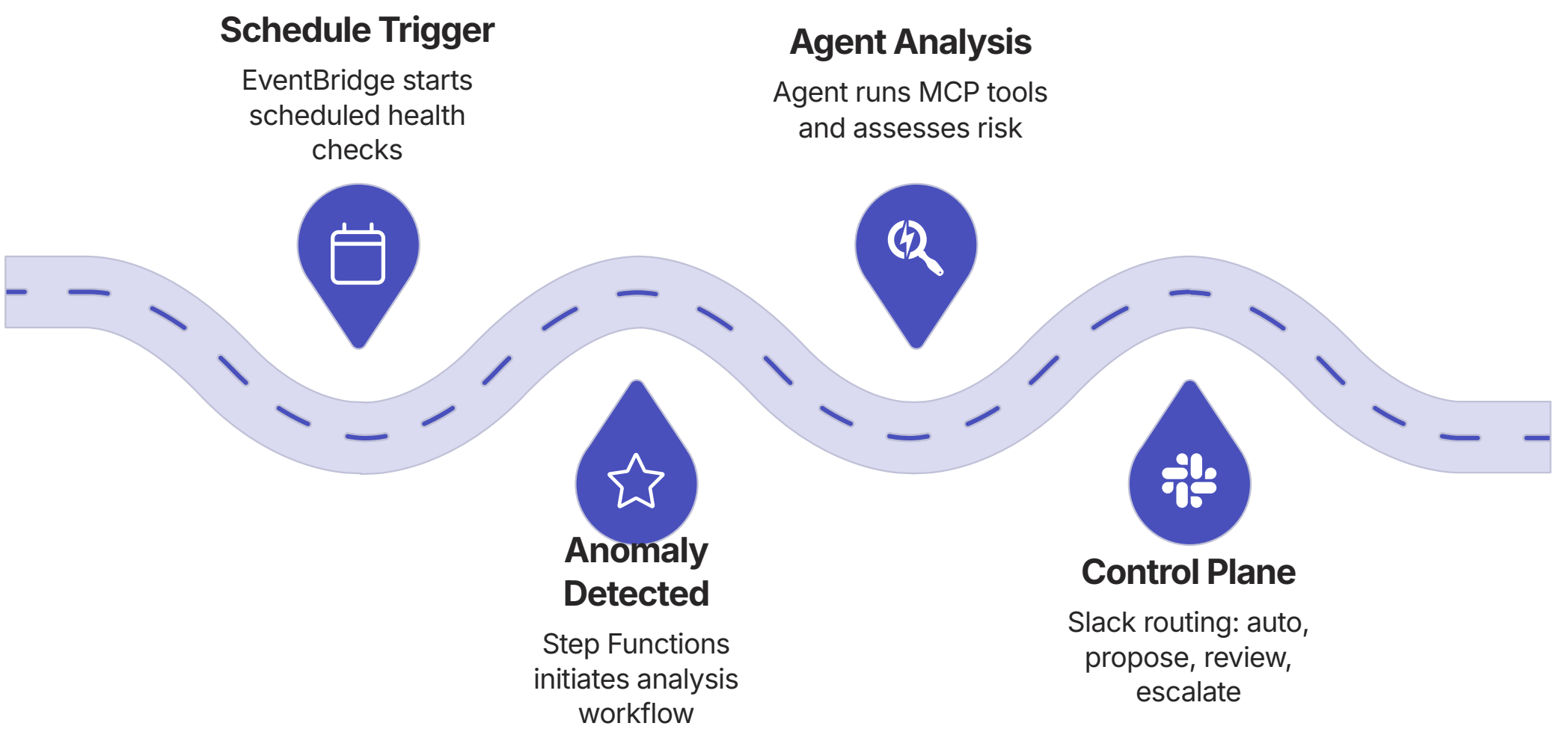
A tool. Reactive. Waits for questions. Only as good as the questions asked.

Proposed Design

"I'm watching your accounts 24/7. I'll tell you when something needs attention and handle it once you say go."

The first is a tool. The second is a team member.

The Always-On Architecture Flow



The architecture should be flipped from request-response to event-driven, continuously monitoring accounts, surfacing alerts, and proposing actions. Slack becomes the **control plane** where the agent surfaces what it found and humans approve or reject actions.

Graduated Autonomy: Not Binary On/Off

Not every action needs approval. The system should have configurable autonomy levels per action type, per client, and per vertical. These thresholds evolve as trust in the system grows.

Autonomy Level	Action Type	Example	Behavior
Auto-execute	Low risk, high confidence	Add negative keyword for obvious garbage term	Execute immediately, log, notify in Slack
Propose & wait	Medium risk	Pause underperforming ad group	Post to Slack, wait for approval via task token
Draft & review	High risk	Launch new campaign, change budget	Create draft, require explicit human review
Escalate	Outside policy	Spend anomaly >50% above normal	Alert Kal directly, block all automated actions on account

Slack as Control Plane. Not Chat Interface

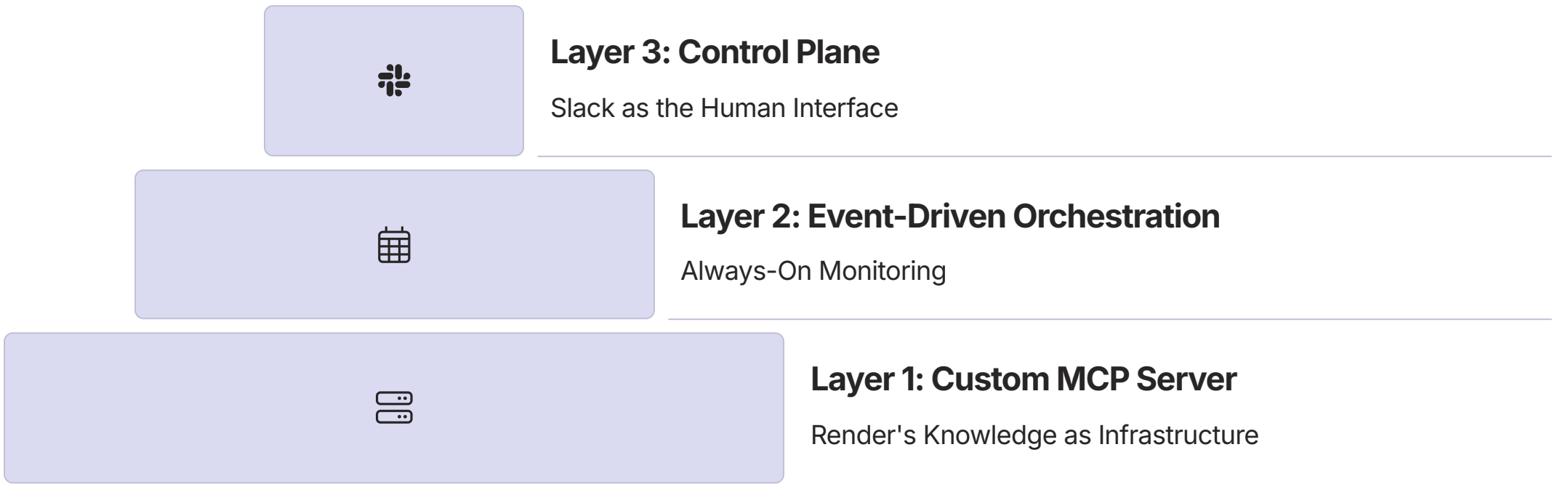
Instead of slash commands that trigger analysis, the agent has already done the analysis. Here's what a production Slack alert looks like:

> **PMS ROAS Alert** — ROAS dropped 34% week-over-week.
> Root cause: 3 new irrelevant search terms eating $412/week.
> Recommended: Add negatives for "mini storage near me free", "storage unit jobs", "storage unit auction".
> **[Approve] [Modify] [Dismiss]**

The human makes the judgment call. The infrastructure executes it. **This is the pattern that separates production AI from demos.**

3. Revised Architecture: Three Layers, Not Three Phases

The strategy document sequences things as: Phase 1 (MCP) → Phase 2 (Agent) → Phase 3 (Slack). This implies you build a chatbot, then add automation later. **Flip it.** Build three layers that work together from the start.



Layer 1: Custom MCP Server

- Wraps Google Ads API with Render's methodology
- Every tool is testable, versioned, deterministic
- This IS the product, the tools are the moat

Layer 2: Event-Driven Orchestration

- EventBridge schedules trigger health checks and anomaly detection
- Step Functions handle: detect → analyze → recommend → approve → execute → log
- The agent doesn't wait for questions. It finds problems.

Layer 3: Control Plane

- Surfaces alerts, proposals, and reports
- Collects approvals via Slack interactive messages → Step Functions task tokens
- Supports on-demand queries. The chatbot use case still exists, it's just not the primary mode
- Per-client channel isolation maintained

Key Insight: In this model, you can ship Layer 1 + Layer 2 with a simple notification channel (even email or SMS) and add Slack polish later. The value comes from the always-on monitoring and the encoded knowledge — not from the chat interface.

4. What This Means for the Build

A direct comparison of the original plan versus the recommended architectural changes across every major decision point:

Original Plan	Recommended Change
Use Google's official MCP server as-is	Build a custom MCP server that wraps Google Ads API with Render's methodology
System prompt encodes agency knowledge	MCP tools encode agency knowledge. Testable, versioned, deterministic
Slack slash commands trigger analysis	Event-driven system triggers analysis automatically; Slack surfaces results
Phase 2 is a "proprietary agent" with a dashboard	Phase 2 is event-driven orchestration with Step Functions
Phase 3 is Slack integration	Slack is a control plane from the start, even if V1 is just a notification channel
Read-only first, then write actions	Read + monitored alerts from day one; write actions gated by graduated autonomy

5. What Doesn't Change

Several elements of the original strategy are correct and should be preserved exactly as designed. These are not up for revision.

Draft Mode

Draft mode before anything goes live. This is correct, keep this. Non-negotiable for production AI.

Per-Client Channel Isolation

Correct architecture for multi-tenant context. Maintain strict per-client channel separation throughout.

Cross-MCC Intelligence

This is the killer feature. The custom MCP server makes it systematic and not ad hoc, but encoded and repeatable.

Ankush on Python/FastAPI

Right stack for this. Python/FastAPI backend is the correct technical choice for the MCP server and orchestration layer.

Competitive Positioning

"Built by practitioners, not an SEO company" is the right message. The custom MCP server makes this claim technically defensible.

Pricing Model

Revisit after internal validation, but directionally sound. Don't let pricing decisions block architecture decisions.

6. Next Step

See the companion document for the sprint-by-sprint build plan, mapped to production AI architecture principles:

Ads Genie Detailed Implementation Plan

The sprint-by-sprint build plan, mapped to production AI architecture principles. Covers Layer 1 through Layer 3 sequencing, tooling decisions, testing strategy, and deployment approach.

This review is informed by the "**AI. But Production" Framework**, a structured methodology for building production-grade AI systems. The detailed implementation plan applies specific principles from this framework to the Ads Genie architecture.

Document Status

DRAFT (For Discussion)

Prepared by Chris Morancie
Fractional CTO / AI Architect

For: Render Analytics

March 2026